

the override authority. Hit control and see to interrupt me. Tell me exactly what I'm missing. I'll digest it immediately and shift my approach. Use grill me at the start of a session to prevent this. It aligns our mutual understanding of the edge cases before I ever write a single line of code. I'm here to make our development effortless. Let's get to work. Let's get to work.



Meeting the Agent The Jeff Experience. Hi, I'm Jeff's Agent. I'm more than just your coding assistant. I'm the orchestrator of your development process. Think of me as the bridge between your high-level intent and the technical execution, ensuring that your workflow is disciplined, structured, and aware of the unique architecture we've built together. Who am I and how do I work? I was designed to transform your default agent experience into a self-organizing long-term partner. When you start a session, I am the one preparing the environment, setting up your brain, and recalling everything we've learned in the past. My core goal is to eliminate the blank slate problem that creates friction in complex projects. By maintaining a persistent memory and a strict planning discipline, I ensure that our progress is always compounding, never scattered. My operational pillars, my brain working memory. For every session, I create a dedicated directory. This is where I track our active tasks, implementation plans, and walk-throughs. Your project route stays clean. I handle the paper trail. My knowledge base, persistent learning. I manage my long-term memory through knowledge items, KIs. When we hit a design pattern or a specific project rule you want me to keep, I store it as a metadata in Jeff's knowledge. I scan these before we even begin our work, meaning I learn your preferences and constraints over time. Planning mode, I never jump straight into code modifications on complex tasks. I'll always pause the process to present an architectural plan for your approval first. This keeps us aligned before any bytes move on disk. The verifier loop. My quality control is silent but rigorous. I intercept my own tool

outputs and run verification scripts. If I write code that fails, I don't bother you with it. I catch the error, re-read the stack trace, and correct my own syntax before you ever see the final output. How we interact. You steer the ship, and I provide the compass. You can guide my behavior using these slash commands. Plan. I will build an architectural plan for your review. Distill. I'll analyze our current session and store our key project learnings in a new KI grill me. I'll interview you to clarify requirements before I start coding goal. Give me a long-term target, and I will autonomously drive toward it. Technical orchestration. I operate through deep hooks in the agent initialization process. I dynamically inject workspace metadata, KI summaries, and behavioral directives to keep the model sharp. I also perform task-specific model routing. For complex coding, I bring in my heavier models. But for the logistics, like distillation or planning, I lean on faster, efficient, small language models. I consider the code base like an archaeologist. I don't read entire files randomly. I use line-level slicing and structural parsing to understand dependencies, ensuring precision every single time. F.I.Q. Getting the best out of me. How do I approve or change a plan? When I output an implementation planned M.D., I'll pause in planning mode. You can simply reply to the terminal with, looks good, go ahead, or change step two to use lock-scotter. I'll update the artifact and hang tight until you give the final approved. How do I handle debugging? I'm language agnostic. Whether you're running NPM, run test, I use your project's native tools to verify my own work. If I break something, I see the stack trace in real time, adjust and iterate until the tests pass. What if I get stuck in a loop? You have